You can verify that the operations count is dominated by equation (6.7.19) and scales as $O(N^{3/2})$ once again. In a real calculation, you should exploit all the symmetries that let you reduce the workload, such as $g_{-m} = g_m^*$ and $\widetilde{P}_l^m[\cos(\pi - \theta)] = (-1)^{l+m}\widetilde{P}_l^m(\cos\theta)$.

Very recently, algorithms for fast Legendre transforms have been developed, similar in spirit to the FFT [3,6,7]. Theoretically, they reduce the forward and inverse spherical harmonic transforms to $O(N\log^2 N)$ problems. However, current implementations [8] are not much faster than the $O(N^{3/2})$ methods above for $N \sim 500$, and there are stability and accuracy problems that require careful attention [9]. Stay tuned!

CITED REFERENCES AND FURTHER READING:

Magnus, W., and Oberhettinger, F. 1949, *Formulas and Theorems for the Functions of Mathematical Physics* (New York: Chelsea), pp. 54ff.[1]

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`, Chapter 8.[2]

Driscoll, J.R., and Healy, D.M. 1994, "Computing Fourier Transforms and Convolutions on the 2–sphere," *Advances in Applied Mathematics*, vol. 15, pp. 202–250.[3]

Muciaccia, P.F., Natoli, P., and Vittorio, N. 1997, "Fast Spherical Harmonic Analysis: A Quick Algorithm for Generating and/or Inverting Full-Sky, High-Resolution Cosmic Microwave Background Anisotropy Maps," *Astrophysical Journal*, vol. 488, pp. L63–66.[4]

Oh, S.P., Spergel, D.N., and Hinshaw, G. 1999, "An Efficient Technique to Determine the Power Spectrum from Cosmic Microwave Background Sky Maps," *Astrophysical Journal*, vol. 510, pp. 551–563, Appendix A.[5]

Healy, D.M., Rockmore, D., Kostelec, P.J., and Moore, S. 2003, "FFTs for the 2-Sphere: Improvements and Variations," *Journal of Fourier Analysis and Applications*, vol. 9, pp. 341–385.[6]

Potts, D., Steidl, G., and Tasche, M. 1998, " Fast and Stable Algorithms for Discrete Spherical Fourier Transforms," *Linear Algebra and Its Applications*, vol. 275-276, pp. 433–450.[7]

Moore, S., Healy, D.M., Rockmore, D., and Kostelec, P.J. 2007+, *SpharmonicKit*. Software at `http://www.cs.dartmouth.edu/~geelong/sphere`.[8]

Healy, D.M., Kostelec, P.J., and Rockmore, D. 2004, "Towards Safe and Effective High-Order Legendre Transforms with Applications to FFTs for the 2-Sphere," *Advances in Computational Mathematics*, vol. 21, pp. 59–105.[9]

6.8 Fresnel Integrals, Cosine and Sine Integrals

6.8.1 Fresnel Integrals

The two Fresnel integrals are defined by

$$C(x) = \int_0^x \cos\left(\frac{\pi}{2}t^2\right)dt, \qquad S(x) = \int_0^x \sin\left(\frac{\pi}{2}t^2\right)dt \tag{6.8.1}$$

and are plotted in Figure 6.8.1.

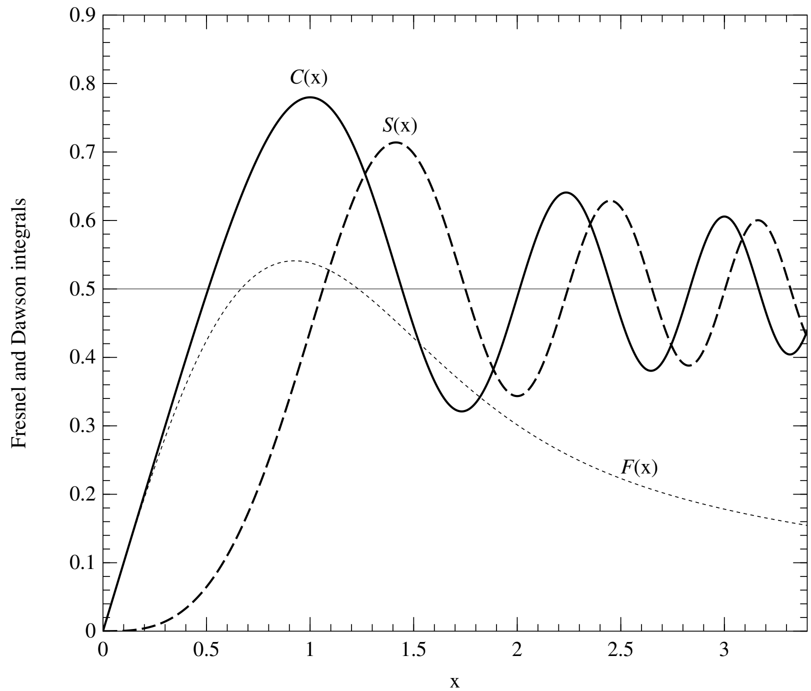

Figure 6.8.1. Fresnel integrals $C(x)$ and $S(x)$ (§6.8), and Dawson's integral $F(x)$ (§6.9).

The most convenient way of evaluating these functions to arbitrary precision is to use power series for small x and a continued fraction for large x. The series are

$$\begin{aligned} C(x) &= x - \left(\frac{\pi}{2}\right)^2 \frac{x^5}{5 \cdot 2!} + \left(\frac{\pi}{2}\right)^4 \frac{x^9}{9 \cdot 4!} - \cdots \\ S(x) &= \left(\frac{\pi}{2}\right) \frac{x^3}{3 \cdot 1!} - \left(\frac{\pi}{2}\right)^3 \frac{x^7}{7 \cdot 3!} + \left(\frac{\pi}{2}\right)^5 \frac{x^{11}}{11 \cdot 5!} - \cdots \end{aligned} \tag{6.8.2}$$

There is a complex continued fraction that yields both $S(x)$ and $C(x)$ simultaneously:

$$C(x) + iS(x) = \frac{1+i}{2} \operatorname{erf} z, \qquad z = \frac{\sqrt{\pi}}{2}(1-i)x \tag{6.8.3}$$

where

$$\begin{aligned} e^{z^2} \operatorname{erfc} z &= \frac{1}{\sqrt{\pi}} \left(\frac{1}{z+} \, \frac{1/2}{z+} \, \frac{1}{z+} \, \frac{3/2}{z+} \, \frac{2}{z+} \cdots \right) \\ &= \frac{2z}{\sqrt{\pi}} \left(\frac{1}{2z^2+1-} \, \frac{1 \cdot 2}{2z^2+5-} \, \frac{3 \cdot 4}{2z^2+9-} \cdots \right) \end{aligned} \tag{6.8.4}$$

In the last line we have converted the "standard" form of the continued fraction to its "even" form (see §5.2), which converges twice as fast. We must be careful not to evaluate the alternating series (6.8.2) at too large a value of x; inspection of the terms shows that $x = 1.5$ is a good point to switch over to the continued fraction.

Note that for large x

$$C(x) \sim \frac{1}{2} + \frac{1}{\pi x} \sin\left(\frac{\pi}{2}x^2\right), \qquad S(x) \sim \frac{1}{2} - \frac{1}{\pi x} \cos\left(\frac{\pi}{2}x^2\right) \tag{6.8.5}$$

Thus the precision of the routine `frenel` may be limited by the precision of the library routines for sine and cosine for large x.

frenel.h

```
Complex frenel(const Doub x) {
```
Computes the Fresnel integrals $S(x)$ and $C(x)$ for all real x. $C(x)$ is returned as the real part of cs and $S(x)$ as the imaginary part.
```
    static const Int MAXIT=100;
    static const Doub PI=3.141592653589793238, PIBY2=(PI/2.0), XMIN=1.5,
        EPS=numeric_limits<Doub>::epsilon(),
        FPMIN=numeric_limits<Doub>::min(),
        BIG=numeric_limits<Doub>::max()*EPS;
```
Here MAXIT is the maximum number of iterations allowed; EPS is the relative error; FPMIN is a number near the smallest representable floating-point number; BIG is a number near the machine overflow limit; and XMIN is the dividing line between using the series and continued fraction.
```
    Bool odd;
    Int k,n;
    Doub a,ax,fact,pix2,sign,sum,sumc,sums,term,test;
    Complex b,cc,d,h,del,cs;
    if ((ax=abs(x)) < sqrt(FPMIN)) {          Special case: Avoid failure of convergence
        cs=ax;                                    test because of underflow.
    } else if (ax <= XMIN) {                  Evaluate both series simultaneously.
        sum=sums=0.0;
        sumc=ax;
        sign=1.0;
        fact=PIBY2*ax*ax;
        odd=true;
        term=ax;
        n=3;
        for (k=1;k<=MAXIT;k++) {
            term *= fact/k;
            sum += sign*term/n;
            test=abs(sum)*EPS;
            if (odd) {
                sign = -sign;
                sums=sum;
                sum=sumc;
            } else {
                sumc=sum;
                sum=sums;
            }
            if (term < test) break;
            odd=!odd;
            n += 2;
        }
        if (k > MAXIT) throw("series failed in frenel");
        cs=Complex(sumc,sums);
    } else {                                  Evaluate continued fraction by modified
        pix2=PI*ax*ax;                            Lentz's method (§5.2).
        b=Complex(1.0,-pix2);
        cc=BIG;
        d=h=1.0/b;
        n = -1;
        for (k=2;k<=MAXIT;k++) {
            n += 2;
            a = -n*(n+1);
            b += 4.0;
            d=1.0/(a*d+b);                    Denominators cannot be zero.
            cc=b+a/cc;
            del=cc*d;
            h *= del;
            if (abs(real(del)-1.0)+abs(imag(del)) <= EPS) break;
        }
```

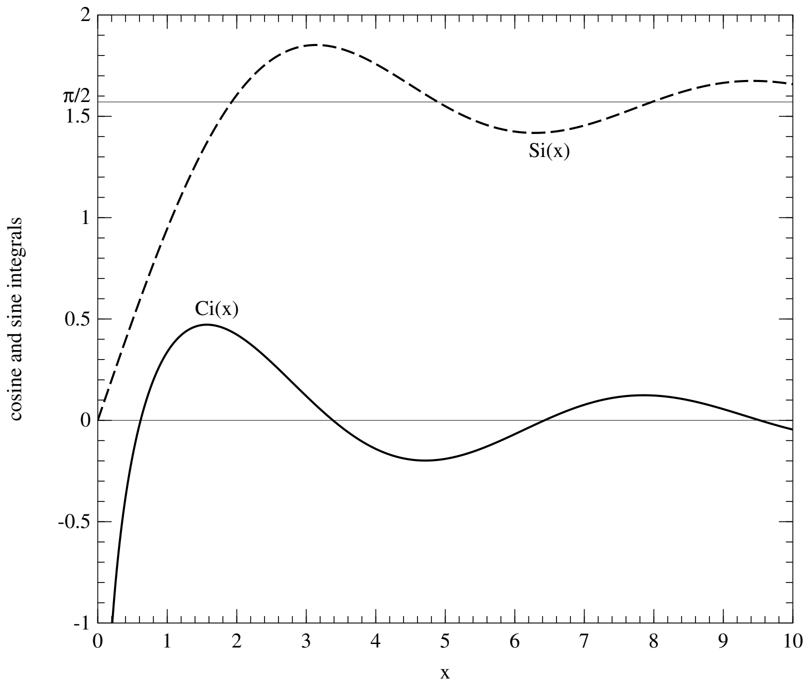

Figure 6.8.2. Sine and cosine integrals $\mathrm{Si}(x)$ and $\mathrm{Ci}(x)$.

```
        if (k > MAXIT) throw("cf failed in frenel");
        h *= Complex(ax,-ax);
        cs=Complex(0.5,0.5)
            *(1.0-Complex(cos(0.5*pix2),sin(0.5*pix2))*h);
    }
    if (x < 0.0) cs = -cs;                          Use antisymmetry.
    return cs;
}
```

6.8.2 Cosine and Sine Integrals

The cosine and sine integrals are defined by

$$\mathrm{Ci}(x) = \gamma + \ln x + \int_0^x \frac{\cos t - 1}{t}\, dt$$
$$\mathrm{Si}(x) = \int_0^x \frac{\sin t}{t}\, dt \tag{6.8.6}$$

and are plotted in Figure 6.8.2. Here $\gamma \approx 0.5772\ldots$ is Euler's constant. We only need a way to calculate the functions for $x > 0$, because

$$\mathrm{Si}(-x) = -\mathrm{Si}(x), \qquad \mathrm{Ci}(-x) = \mathrm{Ci}(x) - i\pi \tag{6.8.7}$$

Once again we can evaluate these functions by a judicious combination of power series and complex continued fraction. The series are

$$\begin{aligned}\mathrm{Si}(x) &= x - \frac{x^3}{3\cdot 3!} + \frac{x^5}{5\cdot 5!} - \cdots \\ \mathrm{Ci}(x) &= \gamma + \ln x + \left(-\frac{x^2}{2\cdot 2!} + \frac{x^4}{4\cdot 4!} - \cdots\right)\end{aligned} \tag{6.8.8}$$

The continued fraction for the exponential integral $E_1(ix)$ is

$$\begin{aligned}E_1(ix) &= -\mathrm{Ci}(x) + i[\mathrm{Si}(x) - \pi/2] \\ &= e^{-ix}\left(\frac{1}{ix+}\,\frac{1}{1+}\,\frac{1}{ix+}\,\frac{2}{1+}\,\frac{2}{ix+}\cdots\right) \\ &= e^{-ix}\left(\frac{1}{1+ix-}\,\frac{1^2}{3+ix-}\,\frac{2^2}{5+ix-}\cdots\right)\end{aligned} \tag{6.8.9}$$

The “even” form of the continued fraction is given in the last line and converges twice as fast for about the same amount of computation. A good crossover point from the alternating series to the continued fraction is $x = 2$ in this case. As for the Fresnel integrals, for large x the precision may be limited by the precision of the sine and cosine routines.

```
Complex cisi(const Doub x) {                                              cisi.h
```

Computes the cosine and sine integrals $\mathrm{Ci}(x)$ and $\mathrm{Si}(x)$. The function $\mathrm{Ci}(x)$ is returned as the real part of cs, and $\mathrm{Si}(x)$ as the imaginary part. $\mathrm{Ci}(0)$ is returned as a large negative number and no error message is generated. For $x < 0$ the routine returns $\mathrm{Ci}(-x)$ and you must supply the $-i\pi$ yourself.

```
    static const Int MAXIT=100;                    Maximum number of iterations allowed.
    static const Doub EULER=0.577215664901533, PIBY2=1.570796326794897,
        TMIN=2.0, EPS=numeric_limits<Doub>::epsilon(),
        FPMIN=numeric_limits<Doub>::min()*4.0,
        BIG=numeric_limits<Doub>::max()*EPS;
```

Here EULER is Euler's constant γ; PIBY2 is $\pi/2$; TMIN is the dividing line between using the series and continued fraction; EPS is the relative error, or absolute error near a zero of $\mathrm{Ci}(x)$; FPMIN is a number close to the smallest representable floating-point number; and BIG is a number near the machine overflow limit.

```
    Int i,k;
    Bool odd;
    Doub a,err,fact,sign,sum,sumc,sums,t,term;
    Complex h,b,c,d,del,cs;
    if ((t=abs(x)) == 0.0) return -BIG;            Special case.
    if (t > TMIN) {                                Evaluate continued fraction by modified
        b=Complex(1.0,t);                              Lentz's method (§5.2).
        c=Complex(BIG,0.0);
        d=h=1.0/b;
        for (i=1;i<MAXIT;i++) {
            a= -i*i;
            b += 2.0;
            d=1.0/(a*d+b);                         Denominators cannot be zero.
            c=b+a/c;
            del=c*d;
            h *= del;
            if (abs(real(del)-1.0)+abs(imag(del)) <= EPS) break;
        }
        if (i >= MAXIT) throw("cf failed in cisi");
        h=Complex(cos(t),-sin(t))*h;
        cs= -conj(h)+Complex(0.0,PIBY2);
    } else {                                       Evaluate both series simultaneously.
```